Esercizi di Ricorsione
Soluzioni degli appelli 2026

Prof. Simone Faro
Dispensa didattica per il corso di Algoritmi e Laboratorio
Università di Catania

Anno 2026

Indice

Introduzione **2**

1 Proprietà lungo un cammino minimo **3**

2 Massima sequenza di nodi neri consecutivi **4**

3 Isomorfismo tra alberi binari **5**

4 Verifica di sottosequenza tra due stringhe **7**

5 Foglie alla stessa profondità **8**

6 Coincidenza tra distanza minima e massima da una foglia **10**

7 Cammini di un DAG con lo stesso nodo terminale **11**

8 Cammini di un DAG con lo stesso costo **13**

Osservazioni conclusive **14**

Introduzione

Questa dispensa raccoglie e svolge i problemi di ricorsione proposti negli appelli di *Algoritmi* dell'anno 2026. Gli esercizi sono presentati in ordine cronologico. Per ciascuno viene riportato il testo del problema, seguito da una soluzione ricorsiva commentata e da una breve analisi della correttezza e del costo computazionale.

Lo scopo non è soltanto fornire il codice finale, ma mettere in evidenza il passaggio più importante nella progettazione di una procedura ricorsiva: individuare quale informazione debba essere restituita da un sottoproblema affinché il livello corrente possa costruire la propria risposta. In alcuni esercizi un valore booleano è sufficiente; in altri è necessario restituire un valore intero oppure una coppia di quantità.

1 Proprietà lungo un cammino minimo

Data dell'appello: 26 gennaio 2026

Problema

Si immagini che i nodi di un grafo direzionato $G \equiv (V, E)$ abbiano associato un vettore $p[v]$, per ogni $v \in V$, che certifica se un nodo v possiede una certa proprietà ($p[v] =$ `True`) o meno ($p[v] =$ `False`). Scrivere una funzione ricorsiva `CheckPath(v,`π`)` che, preso in input il nodo v e l'array π dei predecessori, calcolato da un qualsiasi algoritmo di SSSP, restituisca `True` se tutti i nodi del cammino minimo dalla sorgente a v soddisfano la proprietà p, e `False` altrimenti.

Soluzione

L'array dei predecessori π descrive implicitamente il cammino minimo dalla sorgente a v. Se $\pi[v]$ è il predecessore di v, il cammino dalla sorgente a v è costituito dal cammino dalla sorgente a $\pi[v]$, seguito dal nodo v.

La ricorsione può quindi procedere *all'indietro*, da v verso la sorgente. Prima si controlla il nodo corrente; se esso non soddisfa la proprietà, la risposta è immediatamente falsa. Se invece il nodo la soddisfa, si prosegue sul predecessore. La sorgente è riconoscibile perché il suo predecessore è `NIL`.

Listing 1: Controllo ricorsivo del cammino dei predecessori

```
1 CheckPath(v, pi)
2     if p[v] == False
3         return False
4
5     if pi[v] == NIL
6         return True
7
8     return CheckPath(pi[v], pi)
```

Si assume, come implicito nel testo dell'esercizio, che v sia raggiungibile dalla sorgente. Se si volesse gestire anche il caso di un nodo non raggiungibile, sarebbe necessario distinguere nell'array π la sorgente da un vertice privo di predecessore perché non raggiunto dall'algoritmo SSSP.

Perché la soluzione è corretta

La correttezza segue direttamente dalla struttura del cammino dei predecessori. Se v non soddisfa p, il cammino contiene certamente un nodo che non gode della proprietà e la risposta deve essere falsa. Se v soddisfa p, resta da verificare esattamente la stessa proprietà sul prefisso del cammino che termina in $\pi[v]$. Il caso base viene raggiunto alla sorgente, dove $\pi[v] =$ `NIL`: poiché il nodo corrente è già stato controllato, tutti i nodi del cammino soddisfano p.

Complessità

Se il cammino dalla sorgente a v contiene k nodi, la procedura effettua una chiamata per ciascuno di essi. Il tempo è quindi $\Theta(k)$ e lo spazio occupato dallo stack ricorsivo è $\Theta(k)$. Nel caso peggiore $k \leq |V|$.

2 Massima sequenza di nodi neri consecutivi

Data dell'appello: 16 febbraio 2026

Problema

Si consideri un albero Rosso-Nero, cioè un albero binario di ricerca in cui ogni nodo è colorato rosso o nero e vale il vincolo per cui un nodo rosso non può avere figli rossi, mentre un nodo nero può avere figli sia rossi sia neri. Come di consueto, si assumano presenti le foglie esterne `NIL`, considerate nere. Dato un nodo x dell'albero, si vuole definire e calcolare la seguente quantità:

$$\text{Black-run}(x) = \text{massimo numero di nodi neri consecutivi lungo un cammino discendente da } x \text{ a } \texttt{NIL}.$$

Si richiede di progettare una soluzione ricorsiva che, esplorando i sottoalberi sinistro e destro, calcoli Black-run(x) sfruttando il fatto che la risposta per un nodo può essere espressa in funzione delle risposte dei suoi figli.

Soluzione

In questo problema è importante osservare che il solo valore Black-run di un figlio non contiene tutta l'informazione necessaria. Se x è nero, infatti, vogliamo sapere quanti nodi neri consecutivi possono essere concatenati *a partire dal figlio*. La sequenza massima del figlio potrebbe invece trovarsi più in profondità, dopo un nodo rosso, e quindi non potrebbe essere estesa attraverso x.

Conviene dunque far restituire da ogni chiamata due quantità:

- *start*(x): massimo numero di nodi neri consecutivi che si possono incontrare iniziando proprio da x; vale 0 se x è rosso;
- *best*(x): massimo numero di nodi neri consecutivi che compare in qualunque posizione lungo un cammino da x a una foglia esterna.

La funzione restituisce quindi una coppia (*best*, *start*).

Listing 2: Calcolo ricorsivo di Black-run

```
BlackRunInfo(x)
    if x == NIL
        return (1, 1)

    (bestL, startL) = BlackRunInfo(x.left)
    (bestR, startR) = BlackRunInfo(x.right)

```

```
    if color(x) == BLACK
        start = 1 + max(startL, startR)
    else
        start = 0

    best = max(bestL, bestR, start)
    return (best, start)

BlackRun(x)
    (best, start) = BlackRunInfo(x)
    return best
```

Nel codice si conta la foglia esterna `NIL`, coerentemente con il testo che la considera un nodo nero appartenente al cammino. Se si adottasse la convenzione di non contare il sentinella `NIL` nel valore richiesto, sarebbe sufficiente sostituire il caso base con `return (0,0)`; il resto della ricorrenza rimarrebbe invariato.

Perché la soluzione è corretta

Ogni sequenza massimale di nodi neri nel sottoalbero di x appartiene a uno di tre casi: è interamente contenuta nel sottoalbero sinistro, è interamente contenuta nel sottoalbero destro, oppure inizia in x. I primi due casi sono misurati rispettivamente da *bestL* e *bestR*. Il terzo è misurato da *start*: se x è rosso non può iniziare in x alcuna sequenza di nodi neri, mentre se x è nero la sequenza può proseguire nel figlio che offre il prefisso nero più lungo. Il massimo dei tre valori è quindi esattamente Black-run(x).

Complessità

Ogni nodo del sottoalbero radicato in x viene visitato una sola volta. Se il sottoalbero contiene n_x nodi, il tempo è $\Theta(n_x)$ e lo spazio dello stack è $\Theta(h_x)$, dove h_x è l'altezza del sottoalbero. Nel caso di un albero Rosso-Nero standard, $h_x = O(\log n_x)$.

3 Isomorfismo tra alberi binari

Data dell'appello: 31 marzo 2026

Problema

Si considerino due alberi binari T_1 e T_2, i cui nodi contengono valori interi. Ciascun albero è definito mediante nodi che possiedono un valore e due riferimenti, rispettivamente al figlio sinistro e al figlio destro, eventualmente nulli.

Due alberi si dicono *isomorfi* se è possibile renderli identici scambiando, in uno o più nodi, il figlio sinistro con il figlio destro. In altre parole, gli alberi devono avere la stessa struttura e gli stessi valori nei nodi corrispondenti, a meno di eventuali scambi tra sottoalberi sinistro e destro.

Si richiede di progettare e descrivere una procedura ricorsiva che, dati in input due alberi binari T_1 e T_2, restituisca un valore booleano che indichi se i due alberi sono isomorfi secondo la definizione fornita. La soluzione deve essere espressa in forma di pseudocodice. Facoltativamente, si discuta la complessità della procedura proposta.

Soluzione

Consideriamo due radici x e y. Esistono anzitutto tre casi immediati:

- se entrambe sono nulle, i due alberi sono isomorfi;
- se una sola è nulla, non possono esserlo;
- se i valori memorizzati nelle due radici sono diversi, non possono esserlo.

Se invece le radici contengono lo stesso valore, esistono esattamente due possibilità compatibili con la definizione:

1. non si effettua alcuno scambio in corrispondenza delle radici, per cui sinistra deve corrispondere a sinistra e destra a destra;
2. si effettua uno scambio, per cui sinistra deve corrispondere a destra e destra a sinistra.

Listing 3: Verifica ricorsiva dell'isomorfismo

```
Isomorphic(x, y)
    if x == NIL and y == NIL
        return True

    if x == NIL or y == NIL
        return False

    if x.value != y.value
        return False

    noSwap = Isomorphic(x.left,  y.left) and
             Isomorphic(x.right, y.right)

    if noSwap
        return True

    withSwap = Isomorphic(x.left,  y.right) and
               Isomorphic(x.right, y.left)

    return withSwap
```

Perché la soluzione è corretta

Se le radici sono compatibili, ogni isomorfismo tra i due alberi deve necessariamente scegliere, alla radice, una delle sole due configurazioni possibili: mantenere l'orientamento dei figli oppure scambiarlo. La procedura prova esattamente queste due possibilità e, in ciascun caso, delega ricorsivamente la stessa verifica alle coppie di sottoalberi corrispondenti. I casi base trattano correttamente alberi vuoti e strutture incompatibili. Ne segue, per induzione sull'altezza degli alberi, che la procedura restituisce `True` se e solo se gli alberi sono isomorfi.

Complessità

La presenza delle due possibili corrispondenze può far sì che, quando la prima fallisce soltanto dopo una lunga esplorazione, sia necessario esaminare anche la seconda. In presenza di molti valori uguali il costo può quindi essere sensibilmente superiore a quello di una semplice visita lineare. Una versione con memoizzazione delle coppie di nodi già confrontate evita ricalcoli e limita il numero di sottoproblemi distinti a $O(|T_1|\,|T_2|)$. Nelle istanze in cui la prima corrispondenza scelta ha successo senza ambiguità, il comportamento è invece lineare nel numero di nodi visitati.

4 Verifica di sottosequenza tra due stringhe

Data dell'appello: 27 aprile 2026

Problema

Siano date due stringhe x e y, di lunghezze rispettivamente n e m, con $n, m \geq 0$. Si richiede di progettare una procedura ricorsiva che stabilisca se la stringa x è una sottosequenza della stringa y, ossia se x può essere ottenuta a partire da y mediante la rimozione di zero o più caratteri, senza alterare l'ordine relativo dei caratteri rimanenti.

La procedura deve essere descritta in pseudocodice e deve avere come parametri di ingresso le due stringhe x e y, insieme alle rispettive lunghezze n e m. In particolare, si richiede di definire una procedura della forma

$$\texttt{isSubsequence}(x, y, n, m)$$

che restituisca un valore booleano indicante se x è o meno una sottosequenza di y. Non è consentito l'uso di costrutti iterativi; la soluzione deve essere formulata esclusivamente mediante l'uso della ricorsione.

Soluzione

La scelta naturale è confrontare gli ultimi caratteri dei prefissi considerati, $x[n-1]$ e $y[m-1]$.

Se $n = 0$, la stringa x è vuota e quindi è sottosequenza di qualunque stringa. Se invece $m = 0$ ma $n > 0$, non ci sono caratteri di y disponibili per costruire x e la risposta è falsa.

Quando gli ultimi caratteri coincidono, possiamo utilizzarli nella sottosequenza e ridurre entrambi i problemi di una posizione. Quando invece sono diversi, il carattere $y[m-1]$ non può rappresentare l'ultimo carattere di x e può essere scartato.

Listing 4: Verifica ricorsiva di sottosequenza

```
isSubsequence(x, y, n, m)
    if n == 0
        return True

    if m == 0
        return False

    if x[n-1] == y[m-1]
```

```
9           return isSubsequence(x, y, n-1, m-1)
10      else
11          return isSubsequence(x, y, n, m-1)
```

Perché la soluzione è corretta

Se $x[n-1] = y[m-1]$, esiste sempre una soluzione che usa questa corrispondenza: resta soltanto da verificare che $x[0 \dots n-2]$ sia sottosequenza di $y[0 \dots m-2]$. Se invece i due caratteri sono diversi, $y[m-1]$ non può essere usato per rappresentare l'ultimo carattere di x; eliminarlo non può quindi distruggere alcuna soluzione valida. I due casi base descrivono esattamente le situazioni in cui la risposta è immediata.

Complessità

A ogni chiamata il valore di m diminuisce di uno; talvolta diminuisce anche n. Si effettuano quindi al più $m+1$ chiamate. Il tempo nel caso peggiore è $\Theta(m)$ e la profondità dello stack è $O(m)$.

5 Foglie alla stessa profondità

Data dell'appello: 22 giugno 2026

Problema

Si consideri un albero binario. Si richiede di scrivere una procedura ricorsiva che, ricevuta come unico parametro la radice di un albero o di un suo sottoalbero, verifichi se tutte le foglie sono poste alla stessa profondità rispetto alla radice considerata. In altre parole, la procedura deve stabilire se tutti i cammini che collegano la radice alle foglie hanno la medesima lunghezza. La procedura deve restituire un valore booleano che indichi se la proprietà è soddisfatta oppure no e non deve utilizzare parametri aggiuntivi oltre alla radice del sottoalbero da analizzare.

Soluzione

Per verificare la proprietà in un'unica visita è utile che la ricorsione calcoli, oltre alla validità del sottoalbero, anche la distanza comune delle sue foglie dalla radice. Questa informazione può essere codificata in un intero restituito da una funzione ausiliaria con un solo parametro:

- un valore $d \geq 0$ significa che tutte le foglie del sottoalbero si trovano alla stessa distanza d dalla radice;
- il valore -1 segnala che la proprietà non è soddisfatta.

La procedura booleana richiesta dall'esercizio si limita quindi a verificare se il risultato della funzione ausiliaria è diverso da -1.

Listing 5: Profondità uniforme delle foglie

```
UniformLeafDepth(x)
    if x == NIL
        return 0

    if x.left == NIL and x.right == NIL
        return 0

    if x.left == NIL
        d = UniformLeafDepth(x.right)
        if d == -1
            return -1
        return d + 1

    if x.right == NIL
        d = UniformLeafDepth(x.left)
        if d == -1
            return -1
        return d + 1

    dl = UniformLeafDepth(x.left)
    dr = UniformLeafDepth(x.right)

    if dl == -1 or dr == -1 or dl != dr
        return -1

    return dl + 1

SameLeafDepth(x)
    return UniformLeafDepth(x) != -1
```

È importante trattare separatamente il caso in cui un nodo possieda un solo figlio: il riferimento nullo non rappresenta una foglia dell'albero e non deve quindi essere confrontato con la profondità delle foglie del sottoalbero non vuoto.

Perché la soluzione è corretta

Per una foglia, la distanza dalla radice del sottoalbero a sé stessa è 0, quindi il valore restituito è corretto. Se esiste un solo figlio, tutte le foglie del nodo corrente appartengono a quel sottoalbero: la proprietà è quindi la stessa del figlio e la loro distanza aumenta di uno. Se esistono entrambi i figli, tutte le foglie sono alla stessa profondità se e solo se entrambi i sottoalberi sono validi e le loro profondità comuni coincidono. La procedura verifica esattamente queste condizioni.

Complessità

Ogni nodo viene visitato una sola volta. Il tempo è $\Theta(n)$ e lo spazio dello stack è $\Theta(h)$, dove h è l'altezza dell'albero.

6 Coincidenza tra distanza minima e massima da una foglia

Data dell'appello: 13 luglio 2026

Problema

Si consideri un albero binario. Definire lo pseudocodice della procedura ricorsiva `min-max-check`(x) che, ricevendo come unico argomento la radice x di un albero o di un suo sottoalbero, verifichi se la distanza minima e la distanza massima tra la radice e una foglia sua discendente coincidono. La procedura deve restituire un valore intero o booleano che indichi se tale proprietà è soddisfatta e non deve utilizzare parametri aggiuntivi oltre alla radice.

Soluzione

La distanza minima e la distanza massima da una foglia coincidono esattamente quando *tutte* le foglie del sottoalbero si trovano alla stessa profondità. Possiamo sfruttare questa equivalenza facendo restituire alla funzione un intero:

- se la proprietà è vera, il valore restituito è la distanza comune dalla radice alle foglie;
- se la proprietà è falsa, viene restituito -1.

In questo modo non è necessario aggiungere alcun parametro alla procedura.

Listing 6: Procedura min-max-check

```
min-max-check(x)
    if x == NIL
        return 0

    if x.left == NIL and x.right == NIL
        return 0

    if x.left == NIL
        d = min-max-check(x.right)
        if d == -1
            return -1
        return d + 1

    if x.right == NIL
        d = min-max-check(x.left)
        if d == -1
            return -1
        return d + 1

    dl = min-max-check(x.left)
    dr = min-max-check(x.right)

    if dl == -1 or dr == -1
```

```
24          return -1
25
26      if dl != dr
27          return -1
28
29      return dl + 1
```

La proprietà richiesta è soddisfatta per la radice r se e solo se

$$\texttt{min-max-check}(r) \neq -1.$$

Perché la soluzione è corretta

Il valore restituito da una chiamata valida rappresenta la distanza comune dalla radice corrente a tutte le sue foglie. Se un nodo ha un solo figlio, tutte le foglie sono necessariamente nel relativo sottoalbero e la distanza aumenta di uno. Se ha due figli, la distanza minima e quella massima coincidono solamente se entrambi i sottoalberi soddisfano la proprietà e restituiscono la stessa distanza. In caso contrario compare almeno una foglia a profondità differente e la funzione restituisce -1.

Complessità

La procedura esegue una visita post-order dell'albero: il tempo è $\Theta(n)$ e lo spazio ausiliario è $\Theta(h)$ per lo stack ricorsivo.

7 Cammini di un DAG con lo stesso nodo terminale

Data dell'appello: 1 settembre 2026

Problema

Si consideri un grafo orientato aciclico (DAG) rappresentato mediante una matrice di adiacenza M di dimensione $n \times n$. Definire lo pseudocodice di una procedura ricorsiva `same-terminal(M,x,n)` che, ricevendo in ingresso la matrice di adiacenza M, un nodo x e il numero n di nodi del grafo, verifichi se tutti i cammini orientati che partono da x e terminano in un nodo terminale conducono allo stesso nodo terminale. Un nodo si considera terminale se non possiede archi uscenti. La procedura deve utilizzare la ricorsione per esplorare il grafo e deve restituire un valore intero opportunamente scelto che consenta di determinare se la proprietà richiesta è soddisfatta.

Soluzione

La scelta naturale consiste nel far restituire alla procedura *l'identificatore del nodo terminale comune*. Se da x tutti i cammini terminano nello stesso vertice t, la funzione restituisce t. Se invece esistono due cammini che conducono a terminali diversi, restituisce un valore speciale `FAIL` che non coincide con alcun identificatore di nodo; assumendo nodi numerati da 1 a n, possiamo scegliere `FAIL = -1`.

Un nodo terminale costituisce il caso base: l'unico cammino che parte da esso e termina in un terminale termina nel nodo stesso, quindi la funzione restituisce x.

Per un nodo non terminale, si calcola ricorsivamente il valore di tutti i suoi successori. Tutti questi valori devono essere validi e identici.

Listing 7: Verifica del terminale comune in un DAG

```
same-terminal(M, x, n)
    terminal = -2          // nessun terminale osservato finora
    hasOutgoing = False

    for v = 1 to n
        if M[x,v] != 0
            hasOutgoing = True
            t = same-terminal(M, v, n)

            if t == -1
                return -1

            if terminal == -2
                terminal = t
            else if terminal != t
                return -1

    if hasOutgoing == False
        return x

    return terminal
```

Qui -2 è soltanto un valore interno che indica che nessun successore è ancora stato esaminato, mentre -1 rappresenta definitivamente il fallimento della proprietà.

Perché la soluzione è corretta

Poiché il grafo è aciclico, ogni catena di chiamate ricorsive raggiunge necessariamente un nodo terminale. Per un terminale la risposta è banalmente il nodo stesso. Consideriamo ora un nodo x non terminale. Ogni cammino da x a un terminale inizia scegliendo uno degli archi (x, v) e prosegue poi lungo un cammino da v a un terminale. Pertanto tutti i cammini da x terminano nello stesso nodo se e solo se, per ogni successore v, tutti i cammini da v hanno un terminale comune e tale terminale è lo stesso per tutti i successori. La procedura controlla esattamente queste condizioni.

Complessità

Nella forma appena scritta uno stesso nodo del DAG può essere raggiunto attraverso cammini diversi e quindi essere ricalcolato più volte. In un DAG con molte convergenze il numero di chiamate può crescere sensibilmente. È possibile evitare completamente questi ricalcoli memorizzando, in un array globale o in una tabella, il risultato già ottenuto per ogni nodo. Con memoizzazione ogni nodo viene risolto una sola volta e, poiché la rappresentazione è una matrice di adiacenza, per ciascun nodo vengono scandite n posizioni: il tempo diventa $O(n^2)$ e lo spazio aggiuntivo $O(n)$, oltre allo stack ricorsivo.

8 Cammini di un DAG con lo stesso costo

Data dell'appello: 21 settembre 2026

Problema

Si consideri un grafo orientato aciclico (DAG) pesato, rappresentato mediante una matrice M di dimensione $n \times n$. Per ogni coppia di nodi u, v, il valore $M[u, v]$ è uguale al peso positivo dell'arco (u, v), se tale arco esiste, e vale 0 altrimenti.

Definire lo pseudocodice di una procedura ricorsiva `same-cost(M,x,n)` che, ricevendo in ingresso la matrice M, un nodo x e il numero n di nodi del grafo, verifichi se tutti i cammini orientati che partono da x e terminano in un nodo terminale hanno lo stesso costo complessivo, dove il costo di un cammino è la somma dei pesi degli archi che lo compongono. Un nodo si considera terminale se non possiede archi uscenti.

La procedura deve utilizzare la ricorsione per esplorare il grafo e deve restituire un valore intero opportunamente scelto che consenta di determinare se la proprietà richiesta è soddisfatta.

Soluzione

Anche in questo caso il valore restituito può contenere contemporaneamente la risposta booleana e l'informazione necessaria al livello superiore. Se tutti i cammini che partono da x e terminano in un nodo terminale hanno lo stesso costo, la funzione restituisce proprio tale costo. Se invece esistono due cammini con costi diversi, restituisce -1.

La scelta di -1 è sicura perché tutti i pesi degli archi sono positivi e quindi ogni costo valido è non negativo.

Per un nodo terminale il costo del cammino vuoto che parte da x e termina in x è 0. Per un nodo non terminale, ogni arco uscente (x, v) genera cammini il cui costo totale è

$$M[x, v] + C(v),$$

dove $C(v)$ è il costo comune restituito ricorsivamente dal successore v. Tutti questi valori devono coincidere.

Listing 8: Verifica ricorsiva dell'uguaglianza dei costi

```
same-cost(M, x, n)
    commonCost = -2       // nessun costo osservato finora
    hasOutgoing = False

    for v = 1 to n
        if M[x,v] > 0
            hasOutgoing = True
            c = same-cost(M, v, n)

            if c == -1
                return -1

            candidate = M[x,v] + c

```

```
            if commonCost == -2
                commonCost = candidate
            else if commonCost != candidate
                return -1

    if hasOutgoing == False
        return 0

    return commonCost
```

La proprietà richiesta è vera per il nodo iniziale x se e solo se il valore restituito da `same-cost(M,x,n)` è diverso da -1.

Perché la soluzione è corretta

Il caso base è corretto perché da un nodo terminale esiste il solo cammino vuoto, di costo 0. Per un nodo non terminale, ogni cammino completo è formato da un primo arco (x, v) e da un cammino che parte da v. Affinché tutti i cammini da x abbiano lo stesso costo sono necessarie e sufficienti due condizioni: per ogni successore v, tutti i cammini da v devono avere già un costo comune; inoltre, i valori $M[x, v] + C(v)$ devono coincidere per tutti i successori. La procedura controlla precisamente queste due condizioni e quindi, per induzione su un ordinamento topologico del DAG, è corretta.

Complessità

Come nell'esercizio precedente, in assenza di memoizzazione un vertice condiviso da più cammini può essere ricalcolato molte volte. Memorizzando il risultato di ciascun nodo, ogni vertice viene valutato una sola volta. Con una matrice di adiacenza si scandiscono n possibili archi uscenti per ognuno degli n nodi, ottenendo tempo $O(n^2)$ e spazio $O(n)$ per la memoria dei risultati, oltre allo stack ricorsivo.

Osservazioni conclusive

Gli esercizi raccolti mostrano alcuni schemi ricorsivi ricorrenti che è utile riconoscere:

- **seguire una catena già definita**, come nell'array dei predecessori di un cammino minimo;
- **combinare le risposte di due sottoalberi**, come negli esercizi sugli alberi binari;
- **restituire più informazione di quella apparentemente richiesta**, come nel calcolo di *Black-run*;
- **usare un valore sentinella** per rappresentare contemporaneamente un risultato e il fallimento di una proprietà;
- **esplorare ricorsivamente un DAG**, sfruttando l'assenza di cicli per garantire la terminazione e la memoizzazione per evitare sottoproblemi ripetuti.

In tutti i casi, il punto centrale è scegliere correttamente il significato del valore restituito dalla chiamata ricorsiva. Una volta individuata tale informazione, il caso base e il passo di combinazione tendono a emergere in modo naturale.